

13주 차 보고서

(2026년 3월 22일 ~ 2026년 3월 28일)

작성자

박신우

이번 주 한 일

03.23 트럭과 플레이어 같이 날아가는 문제 해결

지난주에 트럭이 트렁크에 탑승한채로 여기저기 비비면 트럭과 함께 날아가는 문제가 생겼다.

그것을 해결하기 위해 충돌 박스를 트렁크에 따로 만들기로 결정을 하였다.

트럭의 생성자에서 아래와 같이 박스 컴포넌트를 설정해 주었고.

람다함수로 각 박스들이 다음과 같은 기능을 하도록 하였다.

SetCollisionEnabled(QueryAndPhysics)

→ 충돌 판정이 가능하도록 활성화

SetCollisionResponseToAllChannels(ECR_Ignore)

→ 기본적으로 다른 모든 오브젝트와의 충돌은 무시

SetCollisionResponseToChannel(ECC_Pawn, ECR_Block)

→ 캐릭터(Pawn)만 막도록 설정

SetGenerateOverlapEvents(false)

→ 겹침 이벤트는 발생시키지 않음

```
CargoFloorCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoFloorCollision"));
CargoFloorCollision->SetupAttachment(RootComponent);

CargoLeftWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoLeftWallCollision"));
CargoLeftWallCollision->SetupAttachment(RootComponent);

CargoRightWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoRightWallCollision"));
CargoRightWallCollision->SetupAttachment(RootComponent);

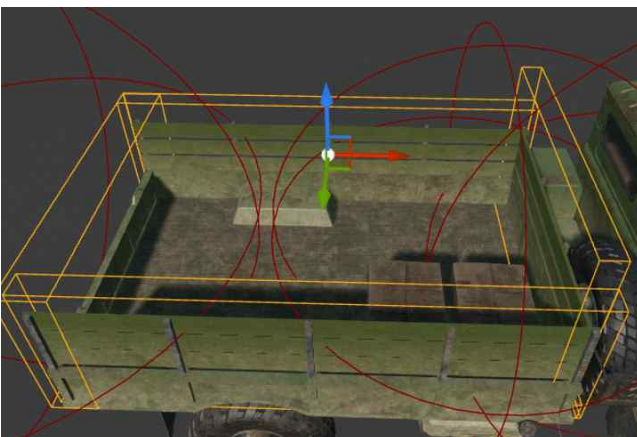
CargoFrontWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoFrontWallCollision"));
CargoFrontWallCollision->SetupAttachment(RootComponent);

CargoBackWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoBackWallCollision"));
CargoBackWallCollision->SetupAttachment(RootComponent);

auto SetupCargoCollision = [](UBoxComponent* Box)
{
    Box->SetCollisionEnabled(ECollisionEnabled::QueryAndPhysics);
    Box->SetCollisionResponseToAllChannels(ECR_Ignore);
    Box->SetCollisionResponseToChannel(ECC_Pawn, ECR_Block);
    Box->SetGenerateOverlapEvents(false);
};

SetupCargoCollision(CargoFloorCollision);
SetupCargoCollision(CargoLeftWallCollision);
SetupCargoCollision(CargoRightWallCollision);
SetupCargoCollision(CargoFrontWallCollision);
SetupCargoCollision(CargoBackWallCollision);
```

에디터에서 다음과 같이 크기를 조절해 주었다.



이번 주 한 일

이제 트렁크에서 특정 범위 밖으로 캐릭터가 나가지 못하게 만들어,
캐릭터랑 트럭이 비벼지며 날아가는 현상이 발생하지 않아졌다.

03.24 트럭 장착 기관총 클래스 생성

트럭에 장착할 기관총 클래스를 생성하였다.

우선, 생성자에서 좌우 회전축, 상하 회전축, 기관총 메쉬, 총알 발사 포인트를 생성하게
하였고,

```
AMountedMachineGun::AMountedMachineGun()
{
    PrimaryActorTick.bCanEverTick = true;

    SceneRoot = CreateDefaultSubobject<USceneComponent>(TEXT("SceneRoot"));
    SetRootComponent(SceneRoot);

    YawPivot = CreateDefaultSubobject<USceneComponent>(TEXT("YawPivot"));
    YawPivot->SetupAttachment(SceneRoot);

    PitchPivot = CreateDefaultSubobject<USceneComponent>(TEXT("PitchPivot"));
    PitchPivot->SetupAttachment(YawPivot);

    GunMesh = CreateDefaultSubobject<USkeletalMeshComponent>(TEXT("GunMesh"));
    GunMesh->SetupAttachment(PitchPivot);
    GunMesh->SetCollisionEnabled(ECollisionEnabled::NoCollision);

    MuzzlePoint = CreateDefaultSubobject<USceneComponent>(TEXT("MuzzlePoint"));
    MuzzlePoint->SetupAttachment(PitchPivot);
    MuzzlePoint->SetRelativeLocation(FVector(150.0f, 0.0f, 0.0f));
}
```

유저 setter를 통해 기관총 사용하는 유저가 누군지 업데이트 시켜주었다.

```
void AMountedMachineGun::SetWeaponUser(AFPSBaseCharacter* NewUser)
{
    CurrentUser = NewUser;
}
```

총알 발사하는 fire 함수에서는 총기 발사 텀을 정해주었다.

```
// 무작정 연사가 안되도록 연사 텀 설정
const float CurrentTime = GetWorld()->GetTimeSeconds();
if (CurrentTime - LastFireTime < FireInterval)
{
    return;
}
LastFireTime = CurrentTime;
```

이번 주 한 일

총알이 발사되는 방향을 결정하기 위해,
플레이어의 눈 위치와 방향을 가져와주었고,

```
FVector CameraLocation;  
FRotator CameraRotation;  
CurrentUser->GetActorEyesViewPoint(CameraLocation, CameraRotation);
```

그 다음,

플레이어가 바라보는 방향으로 10000까지 거리의 선을 쏜다.

그 라인트레이스로 맞은 지점을 찾는다.

무언가 맞았다면? -> 그 맞은 지점이 목표

아무것도 안맞았다면 -> 먼 거리 끝 점이 목표

```
const FVector TraceStart = CameraLocation;  
const FVector TraceEnd = TraceStart + CameraRotation.Vector() * 10000.0f;  
  
FHitResult HitResult;  
FCollisionQueryParams QueryParams;  
QueryParams.AddIgnoredActor(this);  
QueryParams.AddIgnoredActor(CurrentUser);  
  
// 라인트레이스 하여 맞으면 hit  
const bool bHit = GetWorld()->LineTraceSingleByChannel(HitResult, TraceStart, TraceEnd, ECC_Visibility, QueryParams);  
// 아무것도 안맞았다면, 멀리 쏘버리기  
const FVector TargetLocation = bHit ? HitResult.Location : TraceEnd;
```

총기의 총알 발사 위치는 생성자에서 만들고, 블루프린트에서 조정 가능한 MuzzlePoint다.
총구 위치와, 목표지점을 이용해 플레이어가 바라보는 방향으로 총알이 날아가게 하는
방향 벡터를 구했다.

```
const FVector FireLocation = MuzzlePoint ? MuzzlePoint->GetComponentLocation() : GetActorLocation();  
const FVector FireDirection = (TargetLocation - FireLocation).GetSafeNormal();  
const FRotator FireRotation = FireDirection.Rotation();
```

기존에 만들어둔 풀링 시스템이 되어있으면 풀에서 꺼내서 총알을 쓰고,
아니면 그냥 총알을 생성하도록 하였다.

```
if (UObjectPoolSubsystem* PoolSubsystem = GetWorld()->GetSubsystem<UObjectPoolSubsystem>())  
{  
    if (AActor* PooledActor = PoolSubsystem->SpawnFromPool(ProjectileClass, FireLocation, FireRotation))  
    {  
        if (AFPSProjectile* Projectile = Cast<AFPSProjectile>(PooledActor))  
        {  
            Projectile->SetOwner(this);  
            Projectile->SetInstigator(CurrentUser);  
            Projectile->FireInDirection(FireDirection);  
        }  
    }  
}  
else  
{  
    if (AFPSProjectile* Projectile = GetWorld()->SpawnActor<AFPSProjectile>(ProjectileClass, FireLocation, FireRotation))  
    {  
        Projectile->SetOwner(this);  
        Projectile->SetInstigator(CurrentUser);  
        Projectile->FireInDirection(FireDirection);  
    }  
}
```


이번 주 한 일

기관총이 총알을 잘 쏘는 것을 확인할 수 있었다.

03.25 트럭 장착 기관총 문제 해결

여러 가지 문제가 발생하였다.

1. 기관총 총알이 가끔 트럭을 쏘버림.
2. 총알이 약간 이상한 방향으로 발사됨
3. 기관총을 조종할 때 카메라를 회전해도 기관총이 회전하지 않음.

우선 1번 문제를 해결은 간단하게 하였다.

기관총의 fire 함수에서 라인트레이스 할 때 트럭을 무시하도록 하였고,

```
queryParams.AddIgnoredActor(this);  
queryParams.AddIgnoredActor(CurrentUser);  
if (CurrentUser->CurrentTruck)  
{  
    queryParams.AddIgnoredActor(CurrentUser->CurrentTruck);  
}
```

발사체를 생성한 후에도 충돌처리 안하도록 해주었다.

```
Projectile->CollisionComponent->IgnoreActorWhenMoving(this, true);  
Projectile->CollisionComponent->IgnoreActorWhenMoving(CurrentUser, true);  
if (CurrentUser->CurrentTruck)  
{  
    Projectile->CollisionComponent->IgnoreActorWhenMoving(CurrentUser->CurrentTruck, true);  
}
```

2번 문제는 muzzle point를 내가 좀 위쪽에 설정해줬었다.....

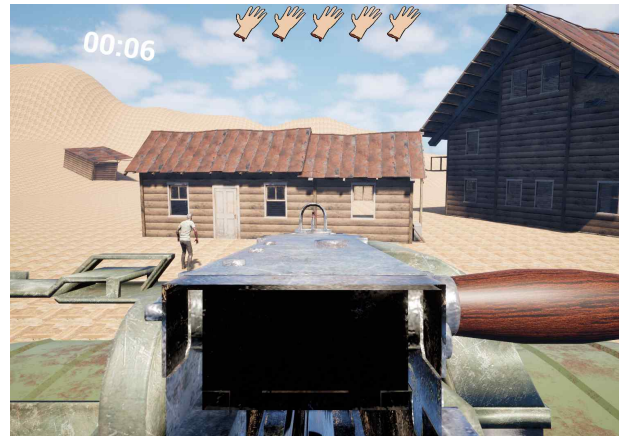
그리고 추가적으로 윤진이가 만든 UI를 기관총에 장착해주었다.



이번 주 한 일

03.26~27 기관총 회전

아래 사진과 같이 정면을 볼 때는 괜찮아보이는데, 회전을 하면 좀 이상한 문제가 있다.



우선, 기존에는 카메라가 기관총 조준선을 따라간 것이 아니라,

기관총의 회전만 따라보는 식이었다.

기관총 생성자에서 기관총의 PitchPivot에 조준용 스프링암과 카메라 포인트를 붙였다.

```
CameraBoom = CreateDefaultSubobject<USpringArmComponent>(TEXT("CameraBoom"));
CameraBoom->SetupAttachment(PitchPivot);
CameraBoom->TargetArmLength = 0.0f;
CameraBoom->bDoCollisionTest = false;
CameraBoom->bUsePawnControlRotation = false;
CameraBoom->bEnableCameraLag = true;
CameraBoom->bEnableCameraRotationLag = true;
CameraBoom->CameraLagSpeed = CameraLocationLagSpeed;
CameraBoom->CameraRotationLagSpeed = CameraRotationLagSpeed;
CameraBoom->SocketOffset = CameraSocketOffset;

CameraPoint = CreateDefaultSubobject<USceneComponent>(TEXT("CameraPoint"));
CameraPoint->SetupAttachment(CameraBoom, USpringArmComponent::SocketName);
```

기관총 회전을 업데이트하는 , UpdateAim() 함수를 만들어주었다.

우선, 기관총의 좌우 회전 목표값을 계산하였다.

이어서 상하 회전 목표값도 계산을 하였다.

아래 코드는 플레이어가 보고 싶은 방향으로 얼마나 돌아야하는 지 계산하고, 최대, 최소 각도 안으로 제한하는 코드이다.

```
// 기관총이 어디를 봐야 하는지 계산
const FRotator ActorRotation = GetActorRotation();
// -120 , 120도 까지만 회전 하도록
// FindDeltaAngleDegrees -> 플레이어 방향과 기관총 방향이 얼마나 차이 나는지 계산
TargetRelativeYaw = FMath::Clamp(FMath::FindDeltaAngleDegrees(ActorRotation.Yaw, ControlRotation.Yaw), MinYaw, MaxYaw);
TargetRelativePitch = FMath::ClampAngle(ControlRotation.Pitch, MinPitch, MaxPitch);
```

이번 주 한 일

플레이어 방향으로 기관총이 바로 바로 회전하면 끊어지는 느낌이 들음으로, 보간을 하여 회전하도록 해주었다.

현재값 : CurrentRelative

목표값 : TargetRelative

를 FInterpTo(Current, Target, DeltaSeconds, Speed)

현재값을 목표값으로 부드럽게 이동해주는 보간 함수에 넣어주었다.

```
// 부드럽게 회전하도록 보간을 해줌
const float DeltaSeconds = GetWorld() ? GetWorld()->GetDeltaSeconds() : 0.0f;
CurrentRelativeYaw = FMath::FInterpTo(CurrentRelativeYaw, TargetRelativeYaw, DeltaSeconds, YawInterpSpeed);
CurrentRelativePitch = FMath::FInterpTo(CurrentRelativePitch, TargetRelativePitch, DeltaSeconds, PitchInterpSpeed);
```

이제 회전값을 넣어 주었다.

Yaw 는 좌우만, Pitch는 상하만 회전에 관여하도록 하여, 더욱 거치형 기관총 느낌이 나도록 만들어 주었다.

```
if (YawPivot)
{
    YawPivot->SetRelativeRotation(FRotator(0.0f, CurrentRelativeYaw, 0.0f));
}

if (PitchPivot)
{
    PitchPivot->SetRelativeRotation(FRotator(CurrentRelativePitch, 0.0f, 0.0f));
}
```

카메라 붐도 업데이트를 시켜주어, 기관총을 잘 따라다니도록 만들어 주었다.

```
if (CameraBoom)
{
    CameraBoom->CameraLagSpeed = CameraLocationLagSpeed;
    CameraBoom->CameraRotationLagSpeed = CameraRotationLagSpeed;
    CameraBoom->SocketOffset = CameraSocketOffset;
}
```

03.28 보고서 정리

다음 주 할 일

- 트럭 탑승, 하차 로직 재검토
- 트럭 운전자가 좌석에 앉아서 운전하도록
- 총알이 조금 더 정확히 가도록.
- 소총 쏘인

특이사항

이번주에 좀 열심히 한 거 같아서 뿌듯했다